Server-Sent Events

HTTP server push with Node.js and Express

CSC 667/867 — Internet Applications

The problem: how do clients learn about changes?

Polling

Browser asks repeatedly:

```
GET /status
GET /status
GET /status
```

Many responses contain no new information.

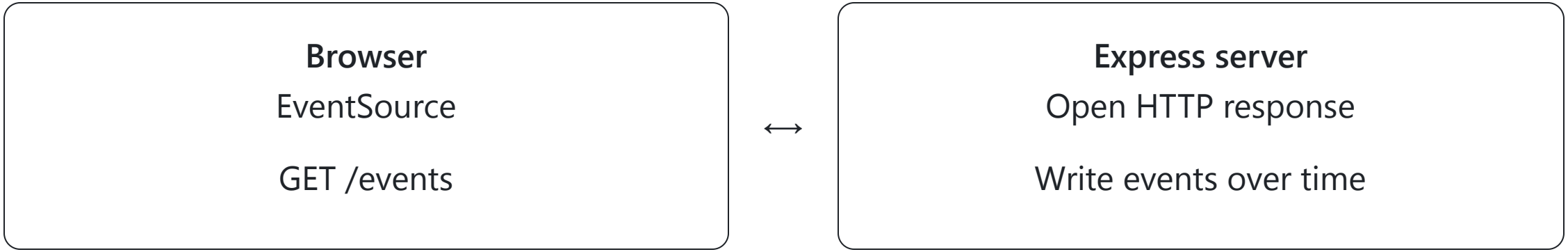
Server-Sent Events

Browser opens one HTTP request:

```
GET /events
  event 1
  event 2
  event 3
```

Server pushes updates as they happen.

SSE: one-way streaming over HTTP



- Server → browser: **SSE stream**, kept open.
- Browser → server: **ordinary HTTP** requests, such as `POST`.
- No WebSocket upgrade; the response is `text/event-stream`.

The SSE wire format

```
HTTP/1.1 200 OK
Content-Type: text/event-stream; charset=utf-8
Cache-Control: no-cache
```

```
retry: 2000

id: 41
event: notice
data: {"text":"Room 12 cleared"}

: heartbeat
```

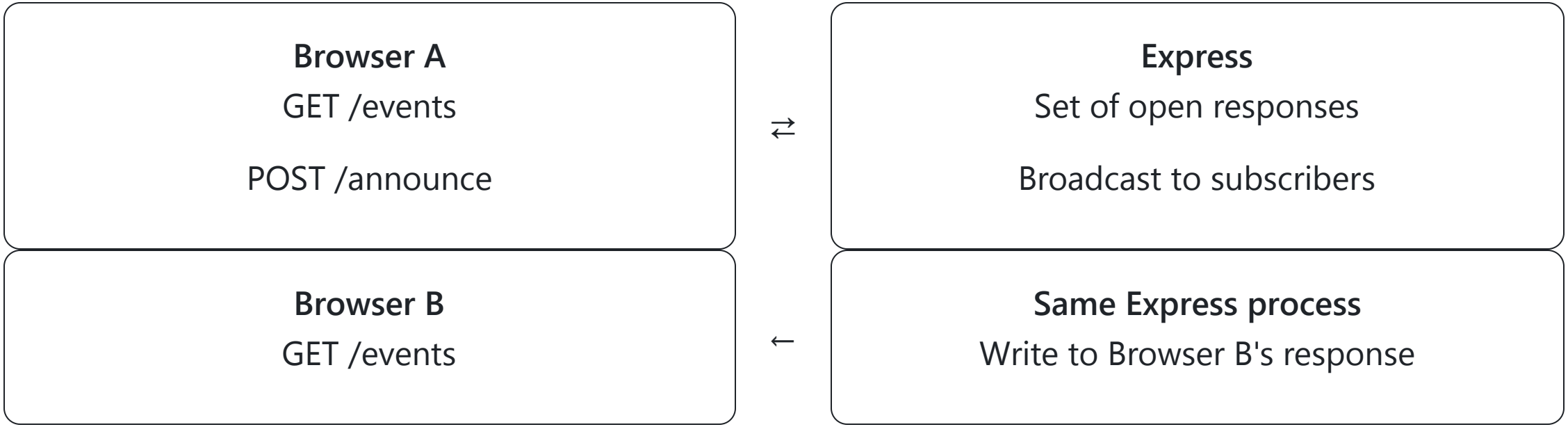
Each event ends with a blank line. The stream remains open.

The event fields

Field	Meaning
`data:`	Message payload as text; JSON is common
`event:`	Named event; otherwise dispatches `message`
`id:`	Updates the client's last event ID
`retry:`	Browser reconnection delay in milliseconds
`:`	Comment; useful for keepalive heartbeats

- Multiple `data:` lines form one payload, joined with newlines.
- Events must be UTF-8 text, separated by a blank line.

Example: announcements to multiple browsers



- The `POST` changes server-side state or triggers an event.
- Every connected SSE client receives the announcement.

Express setup — `server.js`

```
'use strict';

const express = require('express');
const path = require('node:path');

const app = express();
const clients = new Set();
let nextId = 0;

app.use(express.json({ limit: '1kb' }));
app.use(express.static(path.join(__dirname, 'public')));
```

Each client is represented by an **open HTTP response**.

SSE endpoint — `server.js`

```
app.get('/events', (req, res) => {
  res.set({
    'Content-Type': 'text/event-stream; charset=utf-8',
    'Cache-Control': 'no-cache, no-transform',
    'X-Accel-Buffering': 'no'
  });
  res.flushHeaders();
  res.write('retry: 2000\n\n');
  clients.add(res);

  const heartbeat = setInterval(() => res.write(': heartbeat\n\n'), 15000);
  res.on('close', () => {
    clearInterval(heartbeat);
    clients.delete(res);
  });
});
```

`res.write()` streams; `res.end()` would close the connection.

Publish and broadcast — `server.js`

```
app.post('/announce', (req, res) => {
  const text = req.body?.text;
  if (typeof text !== 'string' || !text.trim() || text.length > 200) {
    return res.status(400).json({ error: 'text must be 1-200 characters' });
  }

  const payload = `id: ${++nextId}\nevent: notice\ndata: ${JSON.stringify({ text })}\n\n`;
  for (const client of clients) {
    if (!client.write(payload)) client.end();
  }
  res.json({ id: nextId });
});

app.listen(3000, () => console.log('http://localhost:3000'));
```

`write()` returning `false` signals backpressure; this example closes slow streams.

Browser page — `public/index.html`

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <title>Server-Sent Events Demo</title>
</head>
<body>
  <h1>Announcements</h1>
  <button id="send" disabled>Announce room clear</button>
  <ul id="feed"></ul>
  <script>
    // Client code on the next slide
  </script>
</body>
</html>
```

Browser client — inside `<script>`

```
const source = new EventSource('/events');
const sendButton = document.querySelector('#send');

source.onopen = () => { sendButton.disabled = false; };
source.onerror = () => { sendButton.disabled = true; };

source.addEventListener('notice', event => {
  const li = document.createElement('li');
  li.textContent = JSON.parse(event.data).text;
  document.querySelector('#feed').prepend(li);
});

sendButton.onclick = async () => {
  await fetch('/announce', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ text: 'Room 12 cleared' })
  });
};
```

Run and verify

```
npm install express
node server.js
```

Open `http://localhost:3000` in **two browser tabs**; click the button in either.

Or use two terminals:

```
# Terminal A: observe the open event stream
curl -N http://localhost:3000/events

# Terminal B: publish an event
curl -X POST http://localhost:3000/announce \
  -H 'Content-Type: application/json' \
  -d '{"text":"Room 12 cleared"}'
```

Expected: Terminal A receives an `id`, `event: notice`, and `data:` payload.

Disconnects and reconnection

- `EventSource` **automatically reconnects** after a dropped connection.
- `retry: 2000` requests a 2-second reconnect interval.
- The browser sends `Last-Event-ID` when reconnecting after an event with `id:`.
- `source.close()` stops reconnecting; HTTP `204` can tell the client to stop.
- On the server, `res.on('close')` removes the disconnected client.

A new connection does not automatically receive missed events.

Delivery guarantees are an application decision

Demo behavior

- Events exist only during delivery.
- Disconnected clients miss broadcasts.
- IDs identify events but do not enable replay alone.

For reliable catch-up

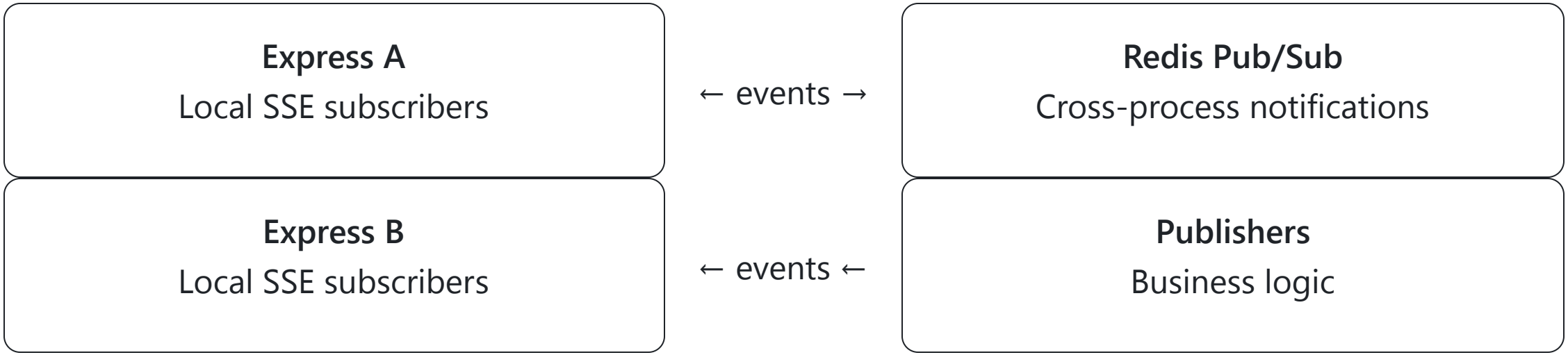
- Store events in an ordered log.
- Read `Last-Event-ID`.
- Replay retained events after that ID.
- Handle duplicates and gaps.

SSE is a transport, not a durable message queue.

Operational requirements

- **Proxies:** disable response buffering; keep `proxy_read_timeout` above heartbeat interval.
- **Compression:** avoid buffered compression or flush each SSE chunk.
- **Backpressure:** bound memory; do not queue indefinitely for slow clients.
- **Security:** authenticate subscriptions and authorize each event stream.
- **Browser limits:** HTTP/1.1 commonly allows only six connections per origin; HTTP/2 multiplexes streams.
- **Authentication:** native `EventSource` cannot attach arbitrary authorization headers; cookies work with appropriate protections.

Scale: one process to many



- Redis is **not required** for SSE within one Node process.
- Redis Pub/Sub distributes events but **does not guarantee replay**.
- Durable replay requires retained history, such as a database or event log.

Choosing a transport

Approach	Direction	Good fit	Main cost
Polling	Client → server	Infrequent status checks	Repeated requests
SSE	Server → client	Notifications, live feeds, job progress	Long-lived connections
WebSocket	Both directions	Games, chat, collaborative editing	Connection and protocol management

Use SSE for server push; use WebSockets for frequent, bidirectional interaction.

Check for understanding

1. Why must the SSE response remain open?
2. Why is `res.write()` used instead of `res.send()`?
3. What happens if a client disconnects during three announcements?
4. What does `Last-Event-ID` make possible, and what else is required?
5. How can a browser send data *to* an SSE server?

Core idea: SSE uses an ordinary HTTP response as a long-lived event stream.

References

1. MDN: Using server-sent events
2. StackPractices: SSE with Node.js and Express
3. Stack Overflow: SSE in Express
4. Baynes: Real-time Web Applications with SSE
5. Express: Compression middleware and SSE